

CONTENTS

- Overview
- 1 Graphs as data: nodes, edges, and the adjacency matrix
- 2 Embeddings, and heterogeneous vs. homogeneous graphs
- 3 Message passing: how nodes share information
- 4 Graph Convolutional Networks (GCN): smoothing over neighbors
- 5 GraphSAGE: sampling neighbors to scale to huge graphs
- 6 Graph Attention Networks (GAT): weighing neighbors unequally
- 7 Graph Isomorphism Networks (GIN) and why sum beats mean/max
- 8 Graph Transformers: global attention over the whole graph
- 9 Choosing an architecture
- Glossary
- Check yourself
- Where to go next

Graph Neural Networks: How GNNs Learn From Connected Data

Understand how message passing works and how GCN, GraphSAGE, GAT, GIN, and graph transformers each aggregate neighbor information differently.

For people comfortable with basic neural networks (weights, activations, matrix multiplication) who want to understand how models learn from graph-structured data · 27 July 2026

[Watch the source video](#)[Read the condensed version](#)[Read the highlights](#)

BEFORE YOU START

- Basic neural network concepts: weights, activation functions, and how backpropagation updates them
- Comfort with matrix multiplication and matrix notation
- Helpful but not required: familiarity with how convolutional neural networks (CNNs) share weights across local regions

Overview

Most machine learning models expect data arranged as a clean table of rows and columns. A lot of real data isn't shaped that way: social networks, molecules, road maps, and citation networks are all collections of entities connected by relationships. A graph captures that directly — entities become nodes, relationships become edges, and a graph neural network (GNN) is a model built to learn from that structure instead of forcing it into a table.

The core mechanism every GNN shares is message passing: each node repeatedly collects information from its neighbors, combines it, and uses it to update its own representation (embedding). Stack enough of these updates and a node's representation ends up encoding not just its own features but the shape of the graph around it.

Where architectures differ is *how* they aggregate neighbor information. This lesson walks through the graph basics (nodes, edges, adjacency matrices, embeddings), the message-passing mechanism itself, and five major architectures — GCN, GraphSAGE, GAT, GIN, and graph transformers — with working code for each aggregation rule.

KEY TAKEAWAYS

- ✓ A graph pairs entities (nodes) with relationships (edges); an adjacency matrix records which nodes connect, and for directed graphs, in which direction.
- ✓ GNNs learn through message passing: every node repeatedly gathers, aggregates, and folds in information from its neighbors to update its own embedding.
- ✓ Stacking message-passing layers grows a node's receptive field — layer 1 reaches immediate neighbors, layer 2 reaches neighbors of neighbors, and so on.
- ✓ GNN architectures mainly differ in their aggregation rule: GCN smooths neighbor features, GraphSAGE samples neighbors to stay scalable, GAT learns per-neighbor attention weights, and GIN sums for maximum expressive power.
- ✓ Sum aggregation (used by GIN) preserves more structural detail than mean or max, which is why GIN matches the discriminative power of the Weisfeiler-Lehman (WL) graph isomorphism test while GCN-style mean/max pooling does not.
- ✓ Graph transformers let every node attend to every other node directly (not just immediate neighbors), and add a structural bias term to the attention score so the graph's topology isn't lost.
- ✓ Choosing an architecture is a tradeoff between expressive power (GIN), scalability to huge graphs (GraphSAGE), interpretable importance weighting (GAT), and long-range reasoning (graph transformers).

01 Graphs as data: nodes, edges, and the adjacency matrix

0:00

A graph represents entities as nodes and their relationships as edges; an adjacency matrix is a simple way to record exactly which nodes connect to which.

Conventional neural networks expect input laid out as fixed-size vectors or grids — a row of features, or a grid of pixels. That works when the data really is that regular, but social circles, molecules, and road networks aren't grids: each entity has a variable number of connections to other entities, and the connections themselves carry information.

A graph G is formally a pair (V, E) : a set of nodes (or vertices) V , and a set of edges E , where each edge is a pair of nodes indicating a connection between them. Edges can be undirected (the relationship goes both ways) or directed (it only goes one way).

A simple, complete way to write down a graph's connectivity is its adjacency matrix: an $N \times N$ matrix (N = number of nodes) where entry (i, j) is 1 if there's an edge from node i to node j , and 0 otherwise. For an undirected graph, the matrix is symmetric. For a directed graph it usually isn't — a student-teacher relationship runs one way, so if node 0 (a student) points to node 1 (a teacher), row 0 / column 1 is 1, but row 1 / column 0 stays 0 because the teacher is not a student of the student.

KEY POINTS

- A graph is a set of nodes plus a set of edges connecting pairs of nodes.
- Undirected edges are symmetric; directed edges point one way and encode asymmetric relationships.
- An adjacency matrix is an $N \times N$ table of 0s and 1s that fully describes which nodes connect to which.
- Row i of the adjacency matrix lists everything node i points to; column i lists everything that points to node i .

Encoding a student-teacher graph

Node 0 is a student of node 1 (a teacher), and also a student of nodes 2 and 4. Because "is student of" is directional, the edges only run from student to teacher, so row 0 has 1s in columns 1, 2, and 4, while row 1 (the teacher) has no outgoing edges back to node 0.

```
import numpy as np

num_nodes = 5
A = np.zeros((num_nodes, num_nodes), dtype=int)

# (student, teacher) pairs -- edge means "is student of"
edges = [(0, 1), (0, 2), (0, 4)]
for src, dst in edges:
    A[src, dst] = 1

print(A)
# row 0 -> [0 1 1 0 1] node 0 is a student of nodes 1, 2, 4
# row 1 -> [0 0 0 0 0] node 1 (a teacher) is a student of no one
print(A[1, 0]) # 0 -- the reverse edge does not exist
```

02 Embeddings, and heterogeneous vs. homogeneous graphs

2:10

GNNs convert nodes (and edges, and whole graphs) into dense embedding vectors, and graphs are classified by whether they contain one type of node/edge or several.

An adjacency matrix is complete but wasteful: most of its entries are 0, and it says nothing about each node's own features (a person's age, a molecule's atom type, and so on). To make graph data usable by a model, GNNs learn embeddings — dense, low-dimensional vectors that capture both a node's own

features and its position in the graph's structure. Embeddings can be computed for nodes, for edges, or for an entire graph, depending on what the downstream task needs (classifying a node, predicting a missing edge, or classifying a whole molecule).

Graphs are also classified by how uniform their nodes and edges are. A homogeneous graph has exactly one type of node and one type of edge — a social network where every node is “person” and every edge is “friend of”. A heterogeneous graph mixes types — the student/teacher graph from the previous section has (at least) two node types (student, teacher) and could have multiple edge types (“is student of”, “teaches”). Heterogeneous GNNs typically need separate weight matrices per node/edge type, since a “student” feature vector and a “teacher” feature vector aren't directly comparable.

KEY POINTS

- An embedding is a dense, low-dimensional vector that summarizes a node's features and its structural context.
- Embeddings can be computed at the node, edge, or whole-graph level depending on the task.
- A homogeneous graph has one node type and one edge type; a heterogeneous graph has more than one of either.
- Heterogeneous GNNs generally need type-specific parameters, because different node/edge types aren't directly comparable.

Homogeneous vs. heterogeneous graph schemas

A friend network is homogeneous: every node is the same type and every edge means the same thing. The student/teacher graph is heterogeneous: two node types and an edge type that only makes sense between them.

```
import numpy as np

# Homogeneous: one node type ("person"), one edge type ("friend_of")
homogeneous_edges = [(0, 1), (1, 2), (2, 3)]

# Heterogeneous: two node types, one edge type restricted to student->teacher
node_types = {0: "student", 1: "teacher", 2: "student", 3: "teacher"}
edges = {(0, 1): "is_student_of", (2, 3): "is_student_of"}

# A node embedding table: one dense vector per node, refined later by message passing
embedding_dim = 4
num_nodes = 4
node_embeddings = np.random.randn(num_nodes, embedding_dim) * 0.01
print(node_embeddings.shape) # (4, 4) -- low-dimensional, dense, one row per node
```

03 Message passing: how nodes share information

2:51

Nodes don't predict in isolation — in each layer, every node sends, aggregates, and folds in messages from its neighbors, and stacking layers widens how far that information travels.

The idea underneath every GNN is message passing. Instead of learning from a node's own features alone, a GNN lets each node exchange information with its neighbors and use that exchange to update its own representation. This happens layer by layer: in layer 1, a node looks only at its immediate (one-hop) neighbors; in layer 2, because those neighbors already absorbed information from their own neighbors in layer 1, the node effectively reaches two hops out. Each additional layer extends the node's receptive field by one more hop, so a deeper GNN captures more global structure while a shallow one stays local.

Concretely, one message-passing layer breaks into three steps. First, message creation: each neighbor packages up something to send — typically its current embedding, possibly combined with edge features like weights or edge type. Second, aggregation: the receiving node combines all incoming messages into a single vector, using an operation like sum, mean, max, or an attention-weighted combination (the choice of aggregation is exactly what distinguishes the architectures covered later in this lesson). Third, update: the node combines the aggregated message with its own previous embedding, usually through a learned weight matrix and a non-linear activation, to produce its new embedding for that layer.

KEY POINTS

- Message passing has three steps per layer: create messages, aggregate them, then update the node's embedding.
- A node's receptive field grows by one hop per layer — layer 1 reaches direct neighbors, layer 2 reaches neighbors of neighbors.
- The aggregation operation (sum, mean, max, attention) is the main design choice that separates GNN architectures.
- Because the aggregation is the same learned function at every node, the model generalizes across nodes it has never explicitly seen it aggregate for.

A generic message-passing layer in NumPy

This implements one layer's worth of create -> aggregate -> update using sum aggregation and a shared weight matrix, on a small 5-node graph. Running it twice shows the receptive field growing from 1-hop to 2-hop neighbors.

```
import numpy as np

def message_passing_layer(H, A, W, activation=lambda x: np.maximum(x, 0)):
    """
    H: (num_nodes, in_dim) node embeddings from the previous layer
    A: (num_nodes, num_nodes) adjacency matrix (1 if neighbor, else 0)
    W: (in_dim, out_dim) shared, learnable weight matrix
    """
    messages = H # step 1: each neighbor's embedding is its message
    aggregated = A @ messages # step 2: sum incoming messages from neighbors
    H_next = activation(aggregated @ W) # step 3: transform + non-linearity
    return H_next

num_nodes, in_dim, out_dim = 5, 3, 3
H0 = np.random.randn(num_nodes, in_dim)
A = np.array([
    [0, 1, 1, 0, 0],
    [1, 0, 0, 1, 0],
    [1, 0, 0, 0, 1],
    [0, 1, 0, 0, 0],
    [0, 0, 1, 0, 0],
])
W = np.random.randn(in_dim, out_dim) * 0.1

H1 = message_passing_layer(H0, A, W) # each node now reflects its 1-hop neighbors
H2 = message_passing_layer(H1, A, W) # each node now reflects up to 2-hop neighbors
```

04 Graph Convolutional Networks (GCN): smoothing over neighbors

5:03

GCNs generalize the weight-sharing idea behind CNNs to graphs: every node gets a smoothed, aggregated version of its neighbors' embeddings, passed through a shared weight matrix and a non-linearity.

A Graph Convolutional Network (GCN) is the classic starting point for GNNs, and works conceptually like a CNN applied to a graph instead of a grid of pixels. For a node v at layer L , the layer first averages (aggregates) the embeddings of v 's neighbors from layer $L-1$, then multiplies the result by a weight matrix

W_L shared by every node in that layer, then applies a non-linear activation σ . That non-linearity matters: without it, stacking layers of aggregation and linear transforms would still collapse into one big linear function, unable to represent complex, non-linear patterns in the graph.

Because every node reuses the same weight matrix at a given layer, a GCN generalizes across the whole graph rather than learning a separate rule per node — the same trick that lets a CNN's filters slide across an entire image. This aggregate-transform-activate pattern makes GCNs a strong default for semi-supervised node classification, where only some nodes have labels and the model needs to spread that signal through the graph's structure.

The standard implementation also adds two refinements beyond a plain neighbor average: self-loops (so a node's own previous embedding counts as one of the things being aggregated) and degree normalization (so nodes with many neighbors don't dominate the aggregation purely by having more connections).

KEY POINTS

- Formula: $h_v^L = \sigma(W_L \cdot \text{aggregate}(\{h_u^{L-1} \text{ for } u \text{ in neighbors}(v)\}))$.
- The weight matrix W_L is shared across all nodes at a given layer, mirroring a CNN's shared filters.
- The non-linearity σ is what lets stacked layers represent non-linear relationships instead of collapsing to one linear map.
- GCNs work well for semi-supervised node classification, where labels are sparse and need to propagate through the graph.

A GCN layer with self-loops and degree normalization

The standard GCN layer (Kipf & Welling) adds self-loops to the adjacency matrix so each node includes its own previous embedding, then normalizes by node degree so high-degree nodes don't dominate purely by having more neighbors.

```
import numpy as np

def gcn_layer(H_prev, A, W, activation=lambda x: np.maximum(x, 0)):
    """
    H_prev: (N, d_in) embeddings from layer L-1
    A: (N, N) adjacency matrix WITHOUT self-loops
    W: (d_in, d_out) learnable weights for this layer
    Implements  $h_L = \text{sigma}(D^{-1/2} (A + I) D^{-1/2} H_{(L-1)} W)$ 
    """
    N = A.shape[0]
    A_hat = A + np.eye(N)           # add self-loops
    deg = A_hat.sum(axis=1)         # degree of each node in A_hat
    D_inv_sqrt = np.diag(1.0 / np.sqrt(deg))
    A_norm = D_inv_sqrt @ A_hat @ D_inv_sqrt # symmetric normalization
    return activation(A_norm @ H_prev @ W)

N, d_in, d_out = 5, 3, 3
H0 = np.random.randn(N, d_in)
A = np.array([
    [0, 1, 1, 0, 0],
    [1, 0, 0, 1, 0],
    [1, 0, 0, 0, 1],
    [0, 1, 0, 0, 0],
    [0, 0, 1, 0, 0],
])
W = np.random.randn(d_in, d_out) * 0.1
H1 = gcn_layer(H0, A, W)
```

05 GraphSAGE: sampling neighbors to scale to huge graphs

6:25

GraphSAGE ("sample and aggregate") learns to aggregate over a fixed-size sample of each node's neighbors instead of the whole neighborhood, keeping cost constant on graphs with millions of nodes.

GraphSAGE takes its name from its two-step approach: sample and aggregate. A plain GCN aggregates over a node's entire neighborhood at every layer; on a graph with millions of nodes, some of those neighborhoods (especially two or three hops out) can be enormous, making full aggregation too

expensive. GraphSAGE fixes this by sampling a fixed-size subset of each node's neighbors at every layer, so the compute cost per node stays constant no matter how large or dense the graph is.

For a node v at layer L , GraphSAGE aggregates the embeddings of the sampled neighbors from layer $L-1$, then concatenates that aggregated vector with v 's own embedding from layer $L-1$ (rather than mixing v 's own features into the aggregation, as GCN effectively does through self-loops). The concatenated vector is passed through a weight matrix W and a non-linearity σ to produce v 's updated embedding.

Keeping the self vector separate, concatenated rather than summed in, lets the model learn a distinct weighting for "what I already knew about myself" versus "what my neighbors just told me", which is part of why GraphSAGE tends to generalize well to nodes and even graphs it has not seen during training.

KEY POINTS

- Formula: $h_v^L = \sigma(W \cdot \text{concat}(h_v^{L-1}, \text{aggregate}(\{h_u^{L-1} \text{ for } u \text{ in } \text{sample}(\text{neighbors}(v))\})))$.
- Sampling a fixed number of neighbors per layer keeps cost constant regardless of graph size.
- Concatenating (rather than summing) the node's own embedding with the aggregated neighbor embedding keeps the two signals separately weighted.
- This makes GraphSAGE well suited to very large graphs, and to inductive settings where the model must handle nodes unseen during training.

Neighbor sampling and a GraphSAGE-style layer

Instead of aggregating a node's full neighbor list, only a fixed-size random sample is used per layer — here 5 neighbors, however many the node actually has.

```
import numpy as np

def sample_neighbors(all_neighbor_ids, sample_size=5, rng=np.random.default_rng()):
    if len(all_neighbor_ids) <= sample_size:
        return np.array(all_neighbor_ids)
    return rng.choice(all_neighbor_ids, size=sample_size, replace=False)

def graphsage_layer(H_prev, node_id, sampled_neighbor_ids, W,
                    activation=lambda x: np.maximum(x, 0)):
    """
    H_prev: (N, d_in) embeddings from layer L-1
    W: (2*d_in, d_out) weights sized for [self ; aggregated_neighbors]
    """
    neighbor_agg = H_prev[sampled_neighbor_ids].mean(axis=0)
    combined = np.concatenate([H_prev[node_id], neighbor_agg])
    return activation(W.T @ combined)

N, d_in, d_out = 6, 4, 4
H0 = np.random.randn(N, d_in)
all_neighbors_of_0 = [1, 2, 3, 4, 5] # imagine this list had 10,000 entries
sampled = sample_neighbors(all_neighbors_of_0, sample_size=3)
W = np.random.randn(2 * d_in, d_out) * 0.1
h0_next = graphsage_layer(H0, 0, sampled, W)
```

06 Graph Attention Networks (GAT): weighing neighbors unequally

7:07

GATs learn an attention coefficient for each neighbor so the model can focus more on the connections that matter, instead of treating every neighbor the same.

GCN and GraphSAGE both treat every neighbor's contribution roughly equally (an unweighted sum or mean). Graph Attention Networks (GAT) drop that assumption: not every neighbor should count the same, so GAT learns an attention coefficient α_{vu} for each neighbor u of node v , and uses it to weight u 's contribution before summing.

For node v at layer L , GAT first transforms every neighbor's embedding from layer $L-1$ with a shared weight matrix W , then computes an attention coefficient α_{vu} for each neighbor u using a learnable scoring function over v and u 's transformed features. Those coefficients are initialized as random learnable

parameters and updated through backpropagation just like any other weight. Crucially, the alphas for a given node's neighbors are all positive and normalized with softmax so they sum to 1 — that keeps the aggregation a weighted average rather than letting it explode in scale as a node gains more neighbors.

Node v 's new embedding is then the sum of α_{vu} times the transformed embedding of each neighbor u , passed through a non-linearity σ . Because the coefficients are learned per edge rather than fixed, GAT can end up focusing heavily on a few especially relevant neighbors and mostly ignoring the rest — useful when a node's neighbors clearly aren't equally informative, like a citation network where one cited paper is central to the topic and another is only tangentially related.

KEY POINTS

- Formula: $h_v^L = \sigma(\sum_u \alpha_{vu} \cdot W \cdot h_u^{(L-1)})$ over neighbors u of v .
- Attention coefficients α_{vu} start as learnable parameters and are updated by backpropagation.
- Softmax normalization keeps each node's alphas positive and summing to 1, so aggregation stays a weighted average.
- GAT is a good fit when some neighbors are clearly more informative than others, and the model should learn which.

Computing attention-weighted neighbor aggregation

For one node v , transform every neighbor's features, score each (v, u) pair with a learned attention vector plus a LeakyReLU, softmax the scores so they sum to 1, then take the weighted sum.

```
import numpy as np

def leaky_relu(x, slope=0.2):
    return np.where(x > 0, x, slope * x)

def gat_layer(H_prev, v_idx, neighbor_ids, W, a,
              activation=lambda x: np.maximum(x, 0)):
    """
    H_prev: (N, d_in) embeddings from layer L-1
    W: (d_in, d_out) shared weight matrix
    a: (2*d_out,) learnable attention vector
    """
    Wh = H_prev @ W # transform every node's features: (N, d_out)

    scores = np.array([
        leaky_relu(a @ np.concatenate([Wh[v_idx], Wh[u]]))
        for u in neighbor_ids
    ])
    exp_scores = np.exp(scores - scores.max()) # numerically stable softmax
    alpha = exp_scores / exp_scores.sum() # sums to 1 across neighbors

    weighted_sum = sum(a_i * Wh[u] for a_i, u in zip(alpha, neighbor_ids))
    return activation(weighted_sum), alpha

N, d_in, d_out = 5, 3, 3
H0 = np.random.randn(N, d_in)
W = np.random.randn(d_in, d_out) * 0.1
a = np.random.randn(2 * d_out) * 0.1
h0_next, alphas = gat_layer(H0, v_idx=0, neighbor_ids=[1, 2, 4], W=W, a=a)
print(alphas, alphas.sum()) # per-neighbor weights, summing to ~1.0
```

07 Graph Isomorphism Networks (GIN) and why sum beats mean/max

8:31

GIN sums neighbor embeddings and pushes the result through an MLP, and that sum is what lets it tell apart graph structures that mean- or max-based GNNs incorrectly treat as identical.

The Graph Isomorphism Network (GIN) is, among common GNNs, the most expressive at telling different graph structures apart, despite using a simpler building block than GCN or GAT: a multilayer perceptron (MLP), a small stack of fully connected layers with non-linear activations. For node v at layer L , GIN sums the embeddings of all of v 's neighbors from layer $L-1$, adds a scaled copy of v 's own previous embedding, and passes that combined vector through an MLP, which adds the non-linearity and expressive power that a plain sum wouldn't have on its own.

The reason this matters is a real weakness in mean- or max-based aggregation, GCN's included. Take two graphs that are the same size, same number of edges, and where every node has the same degree — structurally, they look identical from those coarse statistics, and are what's called isomorphic when they truly are the same graph up to relabeling. But even when two graphs are NOT isomorphic, mean and max aggregation can still map their differing neighborhoods to the same output, because neither operation is injective: multiple different multisets of neighbor embeddings can average or max out to the same value, so the model literally cannot tell the difference downstream.

Sum aggregation avoids this collapse. Summing distinct multisets is far more likely to produce distinct results than averaging or taking the max, because a sum preserves count and magnitude information that mean and max both discard. This is exactly the property that the Weisfeiler-Lehman (WL) test — a classic, purely combinatorial algorithm for checking whether two graphs are structurally identical — relies on, and GIN's sum-then-MLP design was built specifically to match the WL test's discriminative power, something GCN and similar mean/max-based models cannot do in general.

KEY POINTS

- Formula: $h_v^L = \text{MLP}((1 + \epsilon) \cdot h_v^{L-1} + \sum_u h_u^{L-1})$ over neighbors u of v .
- Sum aggregation is injective over multisets in a way mean and max are not, so it preserves more structural information.
- Two graphs are isomorphic if they're structurally identical up to relabeling nodes; GNNs that rely on mean/max pooling can fail to distinguish even non-isomorphic graphs.
- The Weisfeiler-Lehman (WL) test is a classical algorithm for graph isomorphism checking; GIN was designed to match its expressive power.

Sum vs. mean: which aggregation loses information?

Two different neighbor feature sets can average to the exact same vector, making them indistinguishable to a mean-based aggregator — but their sums differ, so a sum-based aggregator (as in GIN) keeps them separable.

```
import numpy as np

# Neighborhood A: two neighbors with these 1-D features
neighborhood_A = np.array([1.0, 3.0])
# Neighborhood B: a different pair of neighbors
neighborhood_B = np.array([2.0, 2.0])

print(neighborhood_A.mean(), neighborhood_B.mean()) # 2.0, 2.0 -- identical!
print(neighborhood_A.sum(), neighborhood_B.sum())   # 4.0, 4.0 -- also identical here

# But add a third, differently-sized neighborhood C to see sum's real advantage:
neighborhood_C = np.array([1.0, 3.0, 2.0]) # one extra neighbor
print(neighborhood_C.mean()) # 2.0 -- STILL collapses onto A and B under mean
print(neighborhood_C.sum())  # 6.0 -- but sum tells it apart, since it reflects count

def gin_layer(H_prev, neighbor_ids, node_id, mlp, epsilon=0.0):
    neighbor_sum = H_prev[neighbor_ids].sum(axis=0)
    combined = (1 + epsilon) * H_prev[node_id] + neighbor_sum
    return mlp(combined)
```

08 Graph Transformers: global attention over the whole graph

11:31

Graph transformers let any node attend to any other node directly, using standard query/key/value attention plus a structural bias term so the graph's topology still shapes the result.

Every architecture so far only lets a node see neighbors reachable within a small number of hops, controlled by the number of layers stacked. Graph transformers relax that: they use global attention, where in principle any node can attend directly to any other node in a single layer, which helps when the important relationship between two nodes is many hops apart or the graph is dense and messy.

The mechanism starts the same way as a standard transformer. Given a matrix H of node embeddings, three learned linear transformations produce queries, keys, and values: $Q = H W_Q$, $K = H W_K$, $V = H W_V$. The query represents what a node is looking for, the key represents what a node offers, and the value is what actually gets passed along once attention decides how much to weight it. Attention scores come from comparing each node's query to every other node's key: $(Q K^T) / \sqrt{d}$, scaled by the square root of the key dimension d to keep the scores from growing too large as dimensionality increases.

The graph-specific twist is a bias term B added to that score before the softmax: $\text{score} = (Q K^T) / \sqrt{d} + B$. B encodes structural information a plain similarity score would otherwise ignore — whether two nodes are actually connected, how many hops apart they are, or what type of edge connects them — so the model keeps the graph's topology in play even though every node can technically attend anywhere. After softmax, the result is a weighted combination of value vectors. Like standard transformers, graph transformers run several attention heads in parallel (each with its own Q , K , V projections), concatenate the heads' outputs, and combine them with a learned output projection W_O . The result then passes through a feed-forward network (two fully connected layers with a non-linearity like ReLU between them), with a residual connection and layer normalization around both the attention step and the feed-forward step to stabilize training in deeper models.

KEY POINTS

- Global attention lets any node attend to any other node in one layer, not just its direct neighbors.
- Standard attention math applies: $Q = HW_Q$, $K = HW_K$, $V = HW_V$, $\text{score} = QK^T / \sqrt{d}$.
- A structural bias term B is added to the score so connectivity, distance, and edge type still influence attention, even though attention is global.
- Multi-head attention, residual connections, and layer normalization work the same way they do in standard (non-graph) transformers.

Structural-bias attention for a graph transformer

The bias matrix B is built from the adjacency matrix here — connected pairs get a small positive bias, disconnected pairs get a negative bias — as a simple stand-in for a learned structural encoding (shortest-path distance or edge type in a full implementation).

```
import numpy as np

def softmax(x):
    e = np.exp(x - x.max(axis=-1, keepdims=True))
    return e / e.sum(axis=-1, keepdims=True)

def graph_transformer_attention(H, Wq, Wk, Wv, B):
    """
    H: (N, d) node embedding matrix
    Wq, Wk, Wv: (d, d_k) learned projections
    B: (N, N) structural bias (e.g. from adjacency, distance, or edge type)
    """
    Q, K, V = H @ Wq, H @ Wk, H @ Wv
    d_k = Q.shape[-1]
    scores = (Q @ K.T) / np.sqrt(d_k) + B # every node can score every other node
    attn = softmax(scores)
    return attn @ V

N, d, d_k = 5, 4, 4
H = np.random.randn(N, d)
A = np.array([
    [0, 1, 1, 0, 0],
    [1, 0, 0, 1, 0],
    [1, 0, 0, 0, 1],
    [0, 1, 0, 0, 0],
    [0, 0, 1, 0, 0],
])
B = np.where(A == 1, 0.5, -0.5) # simple stand-in for a learned structural bias
Wq = np.random.randn(d, d_k) * 0.1
Wk = np.random.randn(d, d_k) * 0.1
Wv = np.random.randn(d, d_k) * 0.1
H_next = graph_transformer_attention(H, Wq, Wk, Wv, B)
```

09 Choosing an architecture

16:03

Every GNN shares the same message-passing skeleton; the choice of architecture comes down to what you're optimizing for — smoothness, scale, interpretability,

| expressiveness, or long-range reasoning.

All five architectures covered here — GCN, GraphSAGE, GAT, GIN, and graph transformers — share the same underlying pattern: create messages, aggregate them, update the node. What changes is the aggregation rule and how far each layer reaches, and that's exactly what should drive picking one over another for a given problem.

GCN smooths neighbor information with a shared weight matrix and works well as a solid default, especially for semi-supervised node classification with modest-sized graphs. GraphSAGE samples neighbors instead of using the full neighborhood, making it the practical choice once a graph has millions of nodes or needs to generalize to nodes it wasn't trained on. GAT learns which neighbors matter most through attention, which is useful both for accuracy and for interpretability, since the attention weights show which connections the model relied on. GIN trades some of that interpretability for the strongest structural discrimination of the group, matching the classical WL test, which matters most for tasks like graph classification where telling structures apart is the whole point. Graph transformers give up strict locality in exchange for the ability to relate any two nodes directly, which pays off on graphs where the relevant connection is many hops away or the local neighborhood alone is too noisy to be informative.

KEY POINTS

- Every architecture follows the same create/aggregate/update skeleton; only the aggregation rule differs.
- GCN: a strong, simple default for smaller graphs and semi-supervised node classification.
- GraphSAGE: scales to huge graphs and generalizes to unseen nodes via neighbor sampling.
- GAT: attention weights make it both accurate and somewhat interpretable.
- GIN: strongest structural expressiveness, closest to matching the WL test.
- Graph transformers: best when relevant relationships span long range or the graph is dense and noisy.

A quick decision guide

If the graph is small to medium and labels are sparse, start with GCN. If it has millions of nodes, use GraphSAGE. If some neighbors are clearly more relevant than others and interpretability matters, use GAT. If the task is telling graph structures apart (graph classification, molecule property prediction), use GIN. If the important signal is far from a node or the graph is dense and irregular, use a graph transformer.

Glossary

Graph

A structure made of nodes (entities) and edges (relationships between pairs of nodes), formally written as $G = (V, E)$.

Node / vertex

A single entity in a graph, such as a person, atom, or web page.

Edge

A connection between two nodes, representing a relationship; can be directed (one-way) or undirected (two-way).

Adjacency matrix

An $N \times N$ matrix where entry (i, j) is 1 if node i connects to node j , and 0 otherwise; a complete numeric encoding of a graph's structure.

Embedding

A dense, low-dimensional vector representation of a node, edge, or graph that captures both its features and its structural context.

Homogeneous graph

A graph with exactly one type of node and one type of edge.

Heterogeneous graph

A graph with more than one type of node and/or edge, such as a graph with both "student" and "teacher" nodes.

Message passing

The core GNN mechanism where each node repeatedly gathers information from its neighbors (create), combines it (aggregate), and folds it into its own representation (update).

Aggregation

The operation that combines multiple incoming neighbor messages into a single vector, e.g. sum, mean, max, or attention-weighted combination.

Graph Convolutional Network (GCN)

A GNN that aggregates neighbor embeddings (typically via a normalized average) and passes the result through a shared weight matrix and non-linearity, analogous to a CNN's shared filters.

GraphSAGE

"Sample and aggregate" — a GNN that aggregates over a fixed-size random sample of each node's neighbors rather than the full neighborhood, so it scales to very large graphs.

Graph Attention Network (GAT)

A GNN that learns a per-neighbor attention coefficient, so some neighbors contribute more to a node's updated embedding than others.

Attention coefficient

A learned, softmax-normalized weight (denoted α) indicating how much one node's message should count when updating another node's embedding.

Graph Isomorphism Network (GIN)

A GNN that sums neighbor embeddings and passes the result through a multilayer perceptron (MLP); designed to match the discriminative power of the Weisfeiler-Lehman test.

Multilayer perceptron (MLP)

A small neural network made of one or more fully connected layers with non-linear activations between them.

Isomorphic graphs

Two graphs that are structurally identical once you allow relabeling their nodes — same connectivity pattern, just different names for the nodes.

Weisfeiler-Lehman (WL) test

A classical, purely combinatorial algorithm for checking whether two graphs are structurally identical (isomorphic), used as a benchmark for how expressive a GNN's aggregation can be.

Graph transformer

A GNN that applies transformer-style global attention, letting any node attend to any other node, with a structural bias term added to the attention score to preserve graph topology.

Query, key, value (Q, K, V)

Three learned projections of a node's embedding used in attention: the query is what a node looks for, the key is what it offers for comparison, and the value is what gets passed along once attention weights are computed.

Multi-head attention

Running several attention computations (heads) in parallel, each with its own Q/K/V projections, then concatenating and combining their outputs.

Residual connection

Adding a layer's input back to its output before further processing, which helps stabilize training in deep networks.

Check yourself

Answer first, then open each one.

Why can't mean or max aggregation reliably tell two different (non-isomorphic) graph neighborhoods apart, even when GIN's sum aggregation can?

Mean and max are not injective over multisets: multiple different sets of neighbor embeddings can average or max out to the exact same value, so the model literally loses the information needed to distinguish them. Summation preserves count and magnitude information that mean and max discard, which is why GIN's sum-then-MLP design gets closer to matching the discriminative power of the Weisfeiler-Lehman test.

Why does GraphSAGE sample a fixed number of neighbors instead of aggregating over a node's entire neighborhood the way GCN does?

On graphs with millions of nodes, a full neighborhood (especially two or more hops out) can be enormous, making full aggregation computationally infeasible. Sampling a fixed-size subset keeps the cost per node constant regardless of graph size, trading a small amount of information for scalability.

In a GAT, why must the attention coefficients for a given node's neighbors be positive and sum to 1?

They're produced by a softmax, which normalizes raw attention scores into a probability-like weighting. That keeps the aggregated message a weighted average of neighbor features rather than a sum whose scale grows with the number of neighbors, so nodes with many neighbors aren't automatically overweighted.

Why is a non-linear activation applied after the weight matrix multiplication in almost every GNN layer (GCN, GraphSAGE, GAT, and GIN's MLP)?

Without a non-linearity, stacking multiple layers of linear aggregation and linear transformation would still collapse mathematically into a single linear function of the input, no matter how many layers were stacked. The non-linearity is what lets each additional layer actually add representational power for capturing complex, non-linear relationships in the graph.

In a graph transformer's attention score, $QK^T/\sqrt{d} + B$, why is the bias term B necessary given that attention is already global?

Plain QK^T similarity only compares node feature vectors and has no notion of the graph's actual connectivity, since attention is computed between every pair of nodes regardless of whether they're connected. The bias term injects structural information — whether two nodes are connected, how far apart they are, or what edge type links them — so the model doesn't discard the graph's topology just because it can attend anywhere.

Why does stacking more message-passing layers change how much of the graph a node's final embedding reflects?

Each layer only lets a node directly absorb information from its immediate (one-hop) neighbors, but since those neighbors already absorbed information from their own neighbors in the previous layer, stacking layers indirectly extends the reach: layer 2 effectively captures two-hop neighbors, layer 3 captures three-hop neighbors, and so on, growing the node's receptive field by one hop per layer.

Where to go next

- Implement a GCN on the Zachary Karate Club dataset with PyTorch Geometric or DGL, and compare its node-classification accuracy against a plain MLP baseline that ignores graph structure.
- Read the original papers: Kipf & Welling (2017) for GCN, Hamilton, Ying & Leskovec (2017) for GraphSAGE, Veličković et al. (2018) for GAT, and Xu et al. (2019) for GIN.
- Work through the Weisfeiler-Lehman test by hand on two small non-isomorphic graphs to see exactly why mean/max-based GNNs can fail to distinguish them.
- Build a small heterogeneous GNN (e.g. with PyTorch Geometric's HeteroData) on a graph with multiple node/edge types, such as a citation network with papers, authors, and venues.
- Read a graph transformer survey or paper (e.g. Graphormer) to see how different implementations define the structural bias term B.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.